

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



COMP 307

Lab Report

on

“Introduction to Linux”

Submitted by

Jatin Madhikarmi (32)

Submitted to

Ms. Rabina Shrestha

Department of Computer Science and Engineering

Submission Date: December 10 , 2025

Table of Contents

1)What is Linux.....	5
2)The Linux Hierarchical File System.....	5
3)Importance of Linux commands in Operating Systems.....	6
Directory and File Navigation.....	8
1. pwd (Print Working Directory).....	8
2. ls (List).....	8
3. ls -a (List All).....	8
4. ls -l (Long Listing Format).....	9
5. cd (Change Directory).....	11
File and Directory Manipulation.....	11
6. mkdir (Make Directory).....	11
7. rmdir (Remove Directory).....	12
8. rm (Remove).....	12
10. touch.....	14
11. cat (Concatenate).....	14
12. nano, vi, jed.....	14
13. cp (Copy).....	18
14. mv (Move).....	18
Searching and System Information.....	19
15. locate.....	19
16. echo.....	19
17. uname -a (Unix Name - All).....	20
18. df -h (Disk Free - Human-readable).....	20
Process Management and Permissions.....	21
19. ps -u \$USER (Process Status - for User).....	21
20. top.....	22
21. chmod (Change Mode).....	22

List of Figures

Figure 1: pwd command.....	7
Figure 2:ls command.....	7
Figure 3:ls-a command.....	8
Figure 4:ls-l command.....	9
Figure 5: cd command.....	10
Figure 6.1: mkdir command.....	10
Figure 6.2: checking mkdir command.....	11
Figure 6.3: Result of the above command.....	11
Figure 7.1: rmdir command.....	11
Figure 7.2: Checking rmdir command.....	11
Figure 7.3: Result of the above command.....	11
Figure 8: rm command.....	12
Figure 9:rm-r command.....	12
Figure 10:touch command.....	13
Figure 11: cat command.....	13
Figure 12.1: nano command.....	14
Figure 12.2: nano terminal.....	14
Figure 12.3: jed command.....	14
Figure 12.4: jed terminal.....	15
Figure 12.5: vi command.....	15
Figure 12.6: vi terminal.....	16
Figure 13: cp command.....	17
Figure 14.1:mv command for move.....	17
Figure 14.2: mv command for rename.....	18
Figure 15: locate command.....	18
Figure 16: echo command.....	18
Figure 17: uname -a command.....	19
Figure 18: df-h command.....	19
Figure 19.1: ps -u \$USER command(General User).....	20
Figure 19.2: ps -u \$CHRONOS command(Specific User).....	21
Figure 20: top command.....	21
Figure 21: chmod command.....	22

1)What is Linux

Linux is a family of open-source Unix-like operating systems based on the **Linux kernel**. It is one of the most prominent examples of free and open-source software collaboration.

- **Operating System (OS):** Linux is the software on a computer that enables applications and users to access the devices on the computer to perform desired functions. The OS is responsible for managing hardware resources, such as the CPU, memory, and disk storage.
- **Kernel:** The Linux kernel is the core component of the operating system. It acts as the main interface between the computer's hardware and its processes.
- **Open Source:** This means the source code for the OS is freely available to anyone. Users can run, study, modify, and redistribute the code.
- **Distribution (Distro):** A Linux distribution is a complete operating system built on the Linux kernel. It includes the kernel, system utilities, a package management system, a collection of applications, and sometimes a graphical desktop environment. Popular distributions include Ubuntu, Fedora, Debian, Mint, and Red Hat Enterprise Linux.

2)The Linux Hierarchical File System.

The Linux file system is organized in a **hierarchical structure**, resembling an inverted tree, with a single **root directory** denoted by a forward slash (*/\$) at the very top. All files and directories are organized under this root. This standard is defined by the **Filesystem Hierarchy Standard (FHS)**.

Directory	Purpose
/ (Root)	The primary hierarchy root of the entire file system.
/bin	Binaries (Essential user command executables, like <code>ls</code> and <code>cat</code>).
/sbin	System Binaries (Essential system administration executables).

/etc	Etc. (Host-specific configuration files).
/home	User home directories (e.g., <code>/home/user1</code>).
/usr	Unix System Resources (Secondary hierarchy for user-related programs and data; contains most user utilities and applications).
/var	Variable files (Files whose content changes often, like logs and mail queues).
/dev	Devices (Contains device files for hardware like hard drives, CD-ROMs, and terminals).
/proc	Processes (Virtual filesystem providing process and system information).
/tmp	Temporary files (Files that are created and deleted constantly).
/opt	Optional add-on application software packages.

3)Importance of Linux commands in Operating Systems.

Linux commands, executed primarily in the **Terminal** (or Command Line Interface/CLI), are critically important for the following reasons:

- **Automation and Scripting:** Commands can be combined into scripts (e.g., Shell scripts) to automate repetitive and complex tasks (backups, deployments, system maintenance) efficiently and reliably.
- **Precision and Power:** The CLI provides fine-grained control over the system, allowing users to perform complex administrative tasks, networking operations, and file manipulation with precision that is often not available or much slower in a Graphical User Interface (GUI).

- **Resource Efficiency:** The CLI uses significantly fewer system resources (CPU, RAM, and graphical rendering) compared to a GUI. This is crucial for servers, embedded systems, and resource-constrained environments where Linux is dominant.
- **Remote Administration:** System administrators typically manage Linux servers remotely via protocols like SSH. Since these servers often run without a GUI, mastery of command-line tools is essential for maintenance, troubleshooting, and configuration.
- **Standardization:** Unix-like systems share a common set of foundational commands, making skills transferable across different distributions and even other operating systems (like macOS).

Linux Commands

Directory and File Navigation

1. pwd (Print Working Directory)

The `pwd` command stands for "Print Working Directory." Its primary purpose is to display the complete, absolute path name of your current location within the Linux file system hierarchy. This is crucial for navigating and understanding your position relative to the root directory (/). If you are executing scripts or referencing files, knowing the output of `pwd` is essential for specifying correct relative or absolute paths.

```
chronos@chronos-VirtualBox:~$ pwd
/home/chronos
chronos@chronos-VirtualBox:~$
```

Figure 1: pwd command

2. ls (List)

The `ls` command is one of the most fundamental tools, used to list the files and subdirectories contained within a directory. When executed without any arguments, it shows the contents of the current working directory. The command is a quick way to inspect a directory, and by default, it sorts the output alphabetically for easy reading.

```
chronos@chronos-VirtualBox:~$ ls
BashScripts  Downloads  hello.txt  passwd.txt  typescript
Books        echo       helloworld.txt  Pictures    usrbininfo
Cprograms    getrichquick.txt  lazydog.txt  Public      Videos
Ctemplate.c  hellomom.txt  lsinfo.txt   script.txt  whatisInfo.txt
Desktop      hellomsg.txt  ls-t.txt     snap
Documents    hellopl.txt   Music        Templates
chronos@chronos-VirtualBox:~$
```

Figure 2: ls command

3. ls -a (List All)

The `-a` option modifies the standard `ls` command to list **all** files and directories, which is critical for system administration and configuration. This is necessary because, by convention, many system and user configuration files are "hidden" by starting their

names with a dot (e.g., .bashrc, .ssh). Using `ls -a` ensures you can see and manage these important hidden items alongside regular files.

```
chronos@chronos-VirtualBox:~$ ls -a
.          Desktop      helloworld.txt  Public
..         Documents    lazydog.txt     script.txt
.bash_history .dotnet       .lessht        snap
.bash_logout Downloads     lsinfo.txt      .ssh
.bashrc     echo          ls-t.txt        .sudo_as_admin_successful
BashScripts getrichquick.txt Music            Templates
Books       hellomom.txt  passwd.txt      typescript
.cache      hellomsg.txt  Pictures        usrbinfo
.config     hellopl.txt  .pki            Videos
Cprograms  hello.txt    .profile        .vscode
Ctemplate.c  hello.txt    .profile        whatisInfo.txt
chronos@chronos-VirtualBox:~$
```

Figure 3: `ls -a` command

4. `ls -l` (Long Listing Format)

The `-l` option provides a comprehensive, **long listing format** of a directory's contents, displaying crucial metadata about each file or directory. This output includes detailed information such as the file permissions (read, write, execute), the number of hard links, the file owner and group, the exact file size in bytes, and the date and time of the last modification. This level of detail is invaluable for managing permissions and troubleshooting access issues.

```

chronos@chronos-VirtualBox:~$ ls -l
total 528
drwxrwxr-x 2 chronos chronos 4096 Oct 18 10:26 BashScripts
drwxrwxr-x 2 chronos chronos 4096 Oct 5 10:43 Books
drwxrwxr-x 3 chronos chronos 4096 Oct 10 19:02 Cprograms
-rw-rw-r-- 1 chronos chronos 70 Oct 7 11:12 Ctemplate.c
drwxr-xr-x 2 chronos chronos 4096 Sep 26 20:24 Desktop
drwxr-xr-x 2 chronos chronos 4096 Sep 26 20:24 Documents
drwxr-xr-x 2 chronos chronos 4096 Oct 5 10:37 Downloads
-rw-rw-r-- 1 chronos chronos 56 Oct 7 10:38 echo
-rw-rw-r-- 1 chronos chronos 63 Oct 10 18:34 getrichquick.txt
-rw-rw-r-- 1 chronos chronos 19950 Oct 8 12:21 hellomom.txt
-rw-rw-r-- 1 chronos chronos 12 Oct 6 19:16 hellomsg.txt
-rw-rw-r-- 1 chronos chronos 10 Oct 6 19:18 helloopl.txt
-rw-rw-r-- 1 chronos chronos 22 Oct 7 14:30 hello.txt
-rw-rw-r-- 1 chronos chronos 0 Oct 7 14:32 helloworld.txt
-rw-rw-r-- 1 chronos chronos 44 Oct 6 19:21 lazydog.txt
-rw-rw-r-- 1 chronos chronos 237630 Oct 6 19:00 lsinfo.txt
-rw-rw-r-- 1 chronos chronos 56 Oct 6 19:07 ls-t.txt
drwxr-xr-x 2 chronos chronos 4096 Sep 26 20:24 Music
-rwxr-xr-x 1 chronos chronos 64152 Oct 8 12:37 passwd.txt
drwxr-xr-x 3 chronos chronos 4096 Dec 10 10:10 Pictures
drwxr-xr-x 2 chronos chronos 4096 Sep 26 20:24 Public
-rw-rw-r-- 1 chronos chronos 8968 Oct 8 12:32 script.txt
drwx----- 7 chronos chronos 4096 Sep 26 21:00 snap
drwxr-xr-x 2 chronos chronos 4096 Sep 26 20:24 Templates
-rw-rw-r-- 1 chronos chronos 86632 Oct 8 12:42 typescript
-rw-rw-r-- 1 chronos chronos 19950 Oct 6 20:15 usrbininfo
drwxr-xr-x 2 chronos chronos 4096 Sep 26 20:24 Videos
-rw-rw-r-- 1 chronos chronos 6498 Oct 8 12:13 whatisInfo.txt
chronos@chronos-VirtualBox:~$ █

```

Figure 4:ls-l command

5. cd (Change Directory)

The `cd` command is used to **change your current working directory** and is the primary tool for navigating the file system. You can move to any directory by specifying its path, whether it's an absolute path starting from the root (/) or a relative path from your current location. Simple shortcuts like `cd ..` move you up one level (to the parent directory), and `cd ~` will quickly take you back to your home directory.

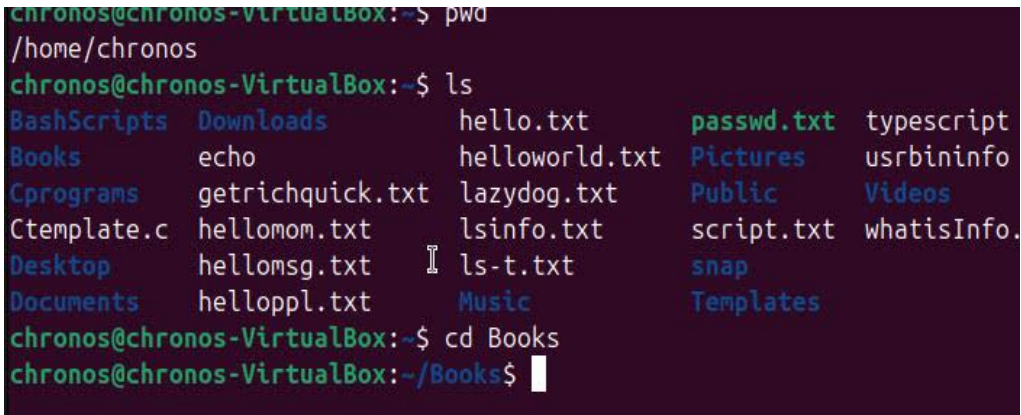
A terminal window showing the execution of the `cd` command. The prompt is `chronos@chronos-VirtualBox:~$`. The user enters `pwd`, and the output is `/home/chronos`. Then, the user enters `ls`, and the output lists various files and directories in a multi-column format: `BashScripts`, `Downloads`, `hello.txt`, `passwd.txt`, `typescript`, `Books`, `echo`, `helloworld.txt`, `Pictures`, `usrbininfo`, `Cprograms`, `getrichquick.txt`, `lazydog.txt`, `Public`, `Videos`, `Ctemplate.c`, `hellomom.txt`, `lsinfo.txt`, `script.txt`, `whatisInfo.`, `Desktop`, `hellomsg.txt`, `ls-t.txt`, `snap`, `Documents`, `hellopl.txt`, `Music`, `Templates`. Finally, the user enters `cd Books`, and the prompt changes to `chronos@chronos-VirtualBox:~/Books$`.

Figure 5: `cd` command

File and Directory Manipulation

6. mkdir (Make Directory)

The `mkdir` command is used to **create a new directory** (or folder) at a specified location. You simply provide the desired name for the new directory as an argument to the command. It is essential for organizing files and creating structured projects within the Linux file system.

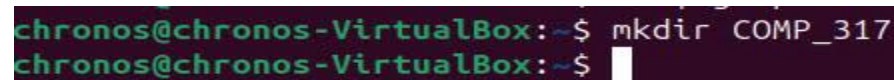
A terminal window showing the execution of the `mkdir` command. The prompt is `chronos@chronos-VirtualBox:~$`. The user enters `mkdir COMP_317`, and the prompt changes to `chronos@chronos-VirtualBox:~$`.

Figure 6.1: `mkdir` command

```
chronos@chronos-VirtualBox:~$ ls | grep COMP | less
```

Figure 6.2: checking mkdir command

```
COMP_317  
(END)
```

Figure 6.3: Result of the above command

7. rmdir (Remove Directory)

The rmdir command is designed to **delete an existing directory** from the file system. A crucial limitation of this command is that it **only works on empty directories**; if the specified directory contains any files or subdirectories, the command will fail and produce an error message. For removing non-empty directories, you must use the more powerful rm -r command instead.

```
chronos@chronos-VirtualBox:~$ rmdir COMP_317  
chronos@chronos-VirtualBox:~$
```

Figure 7.1: rmdir command

```
chronos@chronos-VirtualBox:~$ ls | grep COMP | less
```

Figure 7.2: Checking rmdir command

```
(END)
```

Figure 7.3: Result of the above command

8. rm (Remove)

The rm command is used to **permanently delete files** from the file system. It is a powerful tool, as, unlike deleting in a graphical interface, files removed with rm are

typically not recoverable from a trash bin. Users should exercise caution when using `rm`, especially with wildcards, as the deletion is immediate and permanent.

```
chronos@chronos-VirtualBox:~$ ls
BashScripts  Downloads  hello.txt      passwd.txt     typescript
Books        echo       helloworld.txt Pictures        usrbinfo
Cprograms    getrichquick.txt lazydog.txt    Public         Videos
Ctemplate.c  hellomom.txt lsinfo.txt     script.txt     whatisInfo.txt
Desktop      hellomsg.txt ls-t.txt       snap
Documents    hellopl.txt Music          Templates
chronos@chronos-VirtualBox:~$ rm hello.txt
chronos@chronos-VirtualBox:~$ ls
BashScripts  Downloads  helloworld.txt Pictures        usrbinfo
Books        echo       lazydog.txt    Public         Videos
Cprograms    getrichquick.txt lsinfo.txt     script.txt     whatisInfo.txt
Ctemplate.c  hellomom.txt ls-t.txt       snap
Desktop      hellomsg.txt Music          Templates
Documents    hellopl.txt passwd.txt     typescript
chronos@chronos-VirtualBox:~$
```

Figure 8: `rm` command

9. `rm -r` (Remove Recursively)

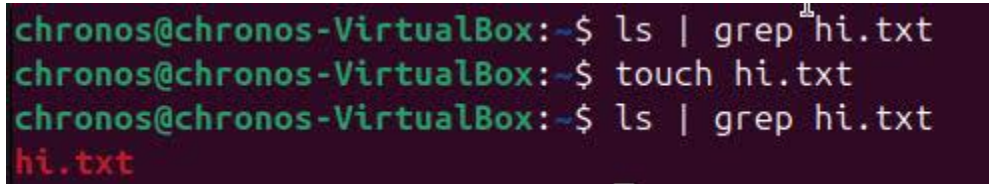
The `-r` (recursive) option modifies the `rm` command to handle directories and their entire contents. This is the command necessary to **delete non-empty directories** as it first deletes all files and subdirectories inside the target directory and then deletes the directory itself. Due to its destructive nature, this combination is the most dangerous command for deleting files and must be used with extreme care.

```
chronos@chronos-VirtualBox:~$ ls | grep Cprograms
Cprograms
chronos@chronos-VirtualBox:~$ cd Cprograms
chronos@chronos-VirtualBox:~/Cprograms$ ls
a.out  hello  hello.c
chronos@chronos-VirtualBox:~/Cprograms$ cd
chronos@chronos-VirtualBox:~$ rm -r Cprograms
chronos@chronos-VirtualBox:~$ ls | grep Cprograms
chronos@chronos-VirtualBox:~$
```

Figure 9: `rm -r` command

10. touch

The touch command serves a dual purpose regarding file timestamps and creation. Its primary use is to **create a new, empty file** when one does not already exist. However, if a file *does* exist, touch updates the file's access and modification timestamps to the current time without altering the file's contents.

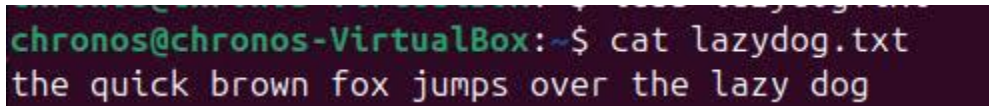
A terminal window with a dark purple background. The prompt is 'chronos@chronos-VirtualBox:~\$'. The first command is 'ls | grep hi.txt', which returns no output. The second command is 'touch hi.txt'. The third command is 'ls | grep hi.txt', which returns 'hi.txt' in red text.

```
chronos@chronos-VirtualBox:~$ ls | grep hi.txt
chronos@chronos-VirtualBox:~$ touch hi.txt
chronos@chronos-VirtualBox:~$ ls | grep hi.txt
hi.txt
```

Figure 10: touch command

11. cat (Concatenate)

The cat command, short for "concatenate," is most often used to **quickly display the contents of one or more files** directly to the terminal screen. While it can join multiple files into a single output stream, its daily use is primarily for viewing the content of small text files. It is a simple and immediate way to inspect file content without needing to open a text editor.

A terminal window with a dark purple background. The prompt is 'chronos@chronos-VirtualBox:~\$'. The command is 'cat lazydog.txt', which outputs the text 'the quick brown fox jumps over the lazy dog' on the next line.

```
chronos@chronos-VirtualBox:~$ cat lazydog.txt
the quick brown fox jumps over the lazy dog
```

Figure 11: cat command

12. nano, vi, jed

These names represent various **text editors** that operate directly within the command-line interface, making them essential tools for configuring Linux systems and writing scripts. **vi** (and its modern version, **vim**) is a classic, highly powerful, but complex modal editor found on virtually all Unix-like systems. **nano** is favored by beginners for its simpler interface and on-screen help menu, while **jed** is another common editor known for its Emacs-like key bindings.

```
chronos@chronos-VirtualBox:~$ nano hello_nano.sh
```

Figure 12.1: nano command



```
GNU nano 7.2 hello_nano.sh *
#!/bin/bash
echo "Hello world"
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line

Figure 12.2: nano terminal

```
chronos@chronos-VirtualBox:~$ jed hello_jed.sh
```

Figure 12.3: jed command

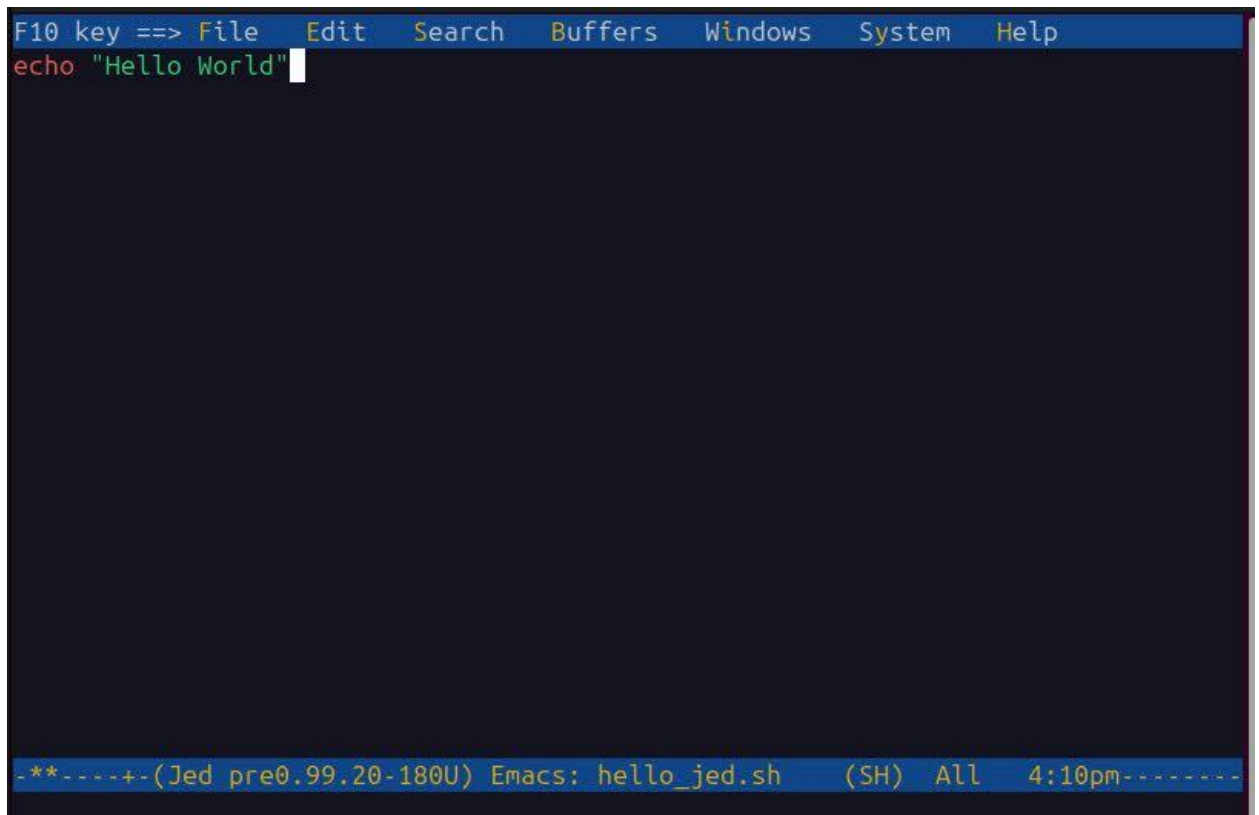


Figure 12.4: jed terminal

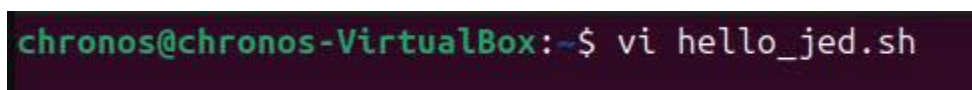


Figure 12.5: vi command

```
echo "Hello World"
```

```
"hello_jed.sh" [Incomplete last line] 1 line, 18 bytes
```

Figure 12.6: vi terminal

13. cp (Copy)

The cp command is used to **copy files and directories** from a source location to a destination location. You must specify the original file (source) and the name and location of the new copy (destination). To copy directories and their contents, you must include the recursive option (-r), much like with the rm command.

```
chronos@chronos-VirtualBox:~$ cd BashScripts
chronos@chronos-VirtualBox:~/BashScripts$ ls
arguments.sh  conditionals.sh  hellomom.sh  hi.sh          trailhelloworld.sh
BashFiles    getrichquick.sh  helloworld.sh  hithere.txt
chronos@chronos-VirtualBox:~/BashScripts$ cp hi.sh hithere.sh
chronos@chronos-VirtualBox:~/BashScripts$ ls
arguments.sh  conditionals.sh  hellomom.sh  hi.sh          hithere.txt
BashFiles    getrichquick.sh  helloworld.sh  hithere.sh    trailhelloworld.sh
```

Figure 13: cp command

14. mv (Move)

The mv command is a versatile utility used for two distinct but related operations: **moving** files or directories to a different location, or **renaming** them. To rename an item, you simply use mv to "move" the file to the same directory but with a new name. Unlike copying, the original file is deleted after the operation is complete.

```
chronos@chronos-VirtualBox:~$ ls | grep Test*
grep: TestDirB: Is a directory
chronos@chronos-VirtualBox:~$ ls | grep *Test
chronos@chronos-VirtualBox:~$ ls | grep Dir
TestDirA
TestDirB
chronos@chronos-VirtualBox:~$ cd TestDirB
chronos@chronos-VirtualBox:~/TestDirB$ ls
morning.sh
chronos@chronos-VirtualBox:~/TestDirB$ cd
chronos@chronos-VirtualBox:~$ mv ./TestDirB/morning.sh ./TestDirA
chronos@chronos-VirtualBox:~$ cd TestDirB
chronos@chronos-VirtualBox:~/TestDirB$ ls
chronos@chronos-VirtualBox:~/TestDirB$ cd
chronos@chronos-VirtualBox:~$ cd TestDirA
chronos@chronos-VirtualBox:~/TestDirA$ ls
morning.sh
```

Figure 14.1: mv command for move

```

chronos@chronos-VirtualBox:~/BashScripts$ ls
arguments.sh  conditionals.sh  hellomom.sh  hihere.txt  hithere.txt
BashFiles    getrichquick.sh  helloworld.sh  hi.sh       trailhelloworld.sh
chronos@chronos-VirtualBox:~/BashScripts$ mv hithere.txt hi.txt
chronos@chronos-VirtualBox:~/BashScripts$ ls
arguments.sh  conditionals.sh  hellomom.sh  hihere.txt  hi.txt
BashFiles    getrichquick.sh  helloworld.sh  hi.sh       trailhelloworld.sh
chronos@chronos-VirtualBox:~/BashScripts$

```

Figure 14.2: mv command for rename

Searching and System Information

15. locate

The locate command is a highly efficient tool used to **search for files by name** across the entire file system. It achieves its speed by querying a pre-built and regularly updated database of all file names, rather than searching the file system in real-time. Because of this reliance on a database, it is extremely fast but can only find files that existed when the database was last updated.

```

chronos@chronos-VirtualBox:~$ locate morning.sh
/home/chronos/TestDirA/morning.sh

```

Figure 15: locate command

16. echo

The echo command is primarily used to **display a line of text or a string** to the standard output, which is usually the terminal. It is a fundamental tool used extensively within shell scripts to output messages to the user, insert text into files, or display the current values of environment variables.

```

chronos@chronos-VirtualBox:~$ echo "Hello world"
Hello world

```

Figure 16: echo command

17. `uname -a` (Unix Name - All)

Function: The `uname -a` command provides a comprehensive summary of the operating system's kernel and hardware information. The `-a` flag stands for "all" and outputs details such as the kernel name (e.g., Linux), the hostname of the system, the kernel release version number, the specific version build, and the machine's hardware architecture. This output is essential for determining compatibility or reporting system details for debugging purposes.

```
chronos@chronos-VirtualBox:~$ uname -a
Linux chronos-VirtualBox 6.14.0-32-generic #32~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC
Tue Sep  2 14:21:04 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
```

Figure 17: `uname -a` command

18. `df -h` (Disk Free - Human-readable)

The `df` command reports the amount of free and used disk space on all mounted filesystems. The `-h` option is critical because it tells the command to output the sizes in a **human-readable** format, using appropriate units like megabytes (M) or gigabytes (G) instead of raw bytes. System administrators use this command constantly to monitor storage capacity and ensure drives do not fill up unexpectedly.

```
chronos@chronos-VirtualBox:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           318M  1.6M  316M   1% /run
/dev/sda2       107G   13G   89G  13% /
tmpfs           1.6G    0  1.6G   0% /dev/shm
tmpfs           5.0M   8.0K  5.0M   1% /run/lock
tmpfs           318M  144K  318M   1% /run/user/1000
/dev/sr0         51M   51M    0 100% /media/chronos/VBox_GAs_7.2.2
```

Figure 18: `df-h` command

Process Management and Permissions

19. ps -u \$USER (Process Status - for User)

The ps command is used to display information about currently running processes. The -u \$USER option filters this list specifically to show only the processes owned by the user currently executing the command (indicated by the \$USER variable). This provides a quick way to monitor the resource usage and status of your personal applications and background tasks.

```
chronos@chronos-VirtualBox:~$ ps -u $USER
  PID TTY          TIME CMD
 1883 ?            00:00:09 systemd
 1887 ?            00:00:00 (sd-pam)
 1896 ?            00:00:00 pipewire
 1897 ?            00:00:00 pipewire
 1899 ?            00:00:02 wireplumber
 1901 ?            00:00:00 pipewire-pulse
 1902 ?            00:00:01 gnome-keyring-d
 1905 ?            00:00:21 dbus-daemon
 1959 tty2          00:00:00 gdm-wayland-ses
 1965 tty2          00:00:00 gnome-session-b
 2012 ?            00:00:01 xdg-document-po
 2026 ?            00:00:00 xdg-permission-
 2058 ?            00:00:00 gcr-ssh-agent
 2060 ?            00:00:00 gnome-session-c
 2077 ?            00:00:00 gvfsd
 2085 ?            00:00:00 gvfsd-fuse
 2088 ?            00:00:04 gnome-session-b
 2121 ?            00:00:00 at-spi-bus-laun
 2132 ?            00:00:01 dbus-daemon
 2133 ?            00:01:04 gnome-shell
 2216 ?            00:00:00 at-spi2-registr
 2234 ?            00:00:00 gnome-shell-cal
 2254 ?            00:00:02 evolution-sourc
 2264 ?            00:00:03 gjs
 2265 ?            00:00:05 Xwayland
 2277 ?            00:00:01 ibus-daemon
 2279 ?            00:00:00 gsd-a11y-settin
 2286 ?            00:00:02 gsd-color
 2291 ?            00:00:00 gsd-datetime
 2298 ?            00:00:00 gsd-housekeepin
```

Figure 19.1: ps -u \$USER command(General User)

```

chronos@chronos-VirtualBox:~$ ps -u $CHRONOS
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
chronos   1959  0.1  0.1 235668  6048 tty2    Ssl+  15:52   0:00 /usr/libexec/gdm-wayland-session env GNOME_SHELL_SESS
chronos   1965  0.3  0.4 298236 16164 tty2    Sl+   15:52   0:00 /usr/libexec/gnome-session-binary --session=ubuntu
chronos   3177  0.2  0.1 11024   5016 pts/0    Ss   15:54   0:00 bash
chronos   3370 100  0.1 13744   4572 pts/0    R+   15:55   0:00 ps -u

```

Figure 19.2: `ps -u $CHRONOS` command(Specific User)

20. top

The `top` command provides a **dynamic, real-time view** of the running system and process activities, making it an essential monitoring tool. It continuously updates the display to show critical system metrics like CPU usage, memory consumption, and swap space. Below the summary, it lists individual processes, ordered by CPU usage, allowing administrators to quickly identify resource-intensive applications.

```

chronos@chronos-VirtualBox:~$ top

top - 15:57:01 up 6 min, 1 user, load average: 1.22, 3.11, 1.75
Tasks: 284 total, 1 running, 283 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.3 us, 0.3 sy, 0.0 ni, 99.1 id, 0.0 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 3175.2 total, 1154.5 free, 1251.1 used, 963.8 buff/cache
MiB Swap: 3175.0 total, 3175.0 free, 0.0 used. 1924.1 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 2133 chronos   20   0 5835652 406284 152988 S   6.0  12.5   2:14.94 gnome-shell
   912 root      20   0 2071584  40488 25356 S   1.3   1.2   0:17.10 snapd
 3396 chronos   20   0  14516    5852  3676 R   1.0   0.2   0:00.18 top
 2094 root      20   0      0      0      0 I   0.7   0.0   0:03.91 kworker/0:3-events
 3098 chronos   20   0 625548   56160 44992 S   0.7   1.7   0:08.74 gnome-terminal-
   471 root      20   0      0      0      0 I   0.3   0.0   0:02.45 kworker/u37:3-events_unbound
    1 root      20   0  23348  13912  9304 S   0.0   0.4   0:24.28 systemd
    2 root      20   0      0      0      0 S   0.0   0.0   0:00.41 kthreadd
    3 root      20   0      0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
    6 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
    7 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
    8 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
    9 root      20   0      0      0      0 I   0.0   0.0   0:03.18 kworker/0:0-rcu_gp
   10 root      20   0      0      0      0 I   0.0   0.0   0:00.05 kworker/0:1-events
   12 root      20   0      0      0      0 I   0.0   0.0   0:00.00 kworker/u32:0-ipv6_addrconf
   13 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
   14 root      20   0      0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
   15 root      20   0      0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
   16 root      20   0      0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread

```

Figure 20: `top` command

21. chmod (Change Mode)

The `chmod` command is used to **change the access permissions** of a file or directory. These permissions define who can read (r), write (w), or execute (x) a file, categorized by

the file's owner (u), the group (g), and others (o). Permissions are typically set using a three-digit octal number (like 755), where each digit represents the permissions for one category.

```
chronos@chronos-VirtualBox:~$ cd TestDirA
chronos@chronos-VirtualBox:~/TestDirA$ ls
morning.sh
chronos@chronos-VirtualBox:~/TestDirA$ ls -l
total 4
-rw-rw-r-- 1 chronos chronos 34 Dec 10 11:15 morning.sh
chronos@chronos-VirtualBox:~/TestDirA$ chmod +x morning.sh
chronos@chronos-VirtualBox:~/TestDirA$ ls -l
total 4
-rwxrwxr-x 1 chronos chronos 34 Dec 10 11:15 morning.sh
```

Figure 21: chmod command